MISUNDERSTOOD — Full Technical Project Description

Project: MISUNDERSTOOD
Tagline: *"Same conversation. Different realities."*
Event: Devlympics — Shunya 2026
Theme: Bridge the Gap
Project type: AI-powered collaboration and communication web application
Team size: 3 members
Development constraint: 24-hour hackathon

1. Project overview

MISUNDERSTOOD is an AI-powered web application that helps people identify and resolve differences in how they understand the same task, instruction, requirement, or goal.

In teamwork, people often agree verbally without realizing that they have different expectations. One person may believe a feature requires email login only, while another assumes both email and phone login are needed. Both may leave the conversation thinking they understand the task, but their interpretations conflict.

MISUNDERSTOOD aims to reveal these gaps **before they become mistakes, wasted effort, or conflicts**.

Instead of allowing participants to see one another's answers immediately, the application first collects each person's independent interpretation. An AI analysis engine then compares the responses and highlights potential contradictions, unconfirmed assumptions, missing requirements, and complementary perspectives.

The application guides participants through clarification questions and helps them create a shared, mutually confirmed understanding of the task.

Core workflow

Create a task → Collect independent interpretations → Detect potential gaps → Ask clarification questions → Resolve differences → Generate a shared agreement → Run the Bridge Test

The objective is not to decide automatically who is right. It is to help people discover where their understandings differ and give them a structured way to resolve those differences.

2. Problem statement

2.1 The problem

Communication failures frequently occur because the same words can mean different things to different people.

For example, a project manager assigns the task:

“Prepare the application for launch.”

Three team members interpret it differently:

- **Developer A:** Finish the login and registration system.
- **Developer B:** Finish the login system, registration system, and password recovery.
- **Designer C:** Complete the interface, onboarding screens, and mobile-responsive design.

Each interpretation is reasonable, but the team has not agreed on the actual scope.

Traditional collaboration tools allow people to communicate, assign tasks, and track progress. However, merely documenting a task does not guarantee that everyone interprets it in the same way.

2.2 Proposed solution

MISUNDERSTOOD introduces a structured process that captures each participant's interpretation independently and compares the responses using AI.

It identifies potential areas of disagreement, displays the evidence behind each finding, asks participants targeted clarification questions, and records the final agreement.

2.3 Problem-to-solution mapping

Problem	MISUNDERSTOOD solution
Different interpretations of the same instruction	Collect independent interpretations
Hidden assumptions	Flag assumptions that have not been confirmed
Conflicting requirements	Highlight specific differences between responses
Missing requirements	Identify important questions that remain
Unproductive clarification conversations	Generate targeted clarification questions
No reliable record of what was agreed	Produce a shared, confirmed checklist
Uncertainty about whether communication improved	Compare unresolved issues before and after clarification

2.4 Theme alignment: Bridge the Gap

The project connects two sides separated by a gap in understanding.

- **Side A:** What one person intends or believes.
- **Side B:** What another person understands or expects.
- **The bridge:** AI-assisted comparison, clarification, and mutual agreement.

This directly interprets the event's *Bridge the Gap* theme as a communication and knowledge gap. The event's design document encourages teams to create a bridge between two sides that struggle to understand or work with one another. It also emphasizes creativity, an enjoyable experience, and a genuinely functional demonstration.

3. Project objectives

The system should achieve the following objectives.

1. **Capture individual understanding:** Allow participants to explain how they interpret a shared task without first seeing other participants' responses.
2. **Detect meaningful differences:** Identify potential contradictions, assumptions, missing details, and complementary ideas.
3. **Explain findings:** Show the exact statements that led to each detected issue.
4. **Facilitate clarification:** Generate specific questions that help participants resolve uncertainties.
5. **Build shared understanding:** Produce a checklist or agreement that participants can review and confirm.
6. **Measure progress:** Compare the issues identified before clarification with those remaining afterward.
7. **Maintain user control:** Let participants correct AI findings, reject incorrect suggestions, and decide what the final agreement should contain.

Non-goals for the hackathon MVP

To keep the project achievable in 24 hours, the first version will not attempt to:

- Record or analyze live meetings.
- Interpret facial expressions, emotions, or body language.
- Automatically determine which participant is correct.
- Integrate with every project-management or messaging platform.
- Provide enterprise identity management or complex permissions.
- Train a custom language model.
- Guarantee that every misunderstanding will be detected.

The MVP will focus on typed responses and a working end-to-end workflow.

4. System users and user journey

4.1 User roles

The MVP needs two logical roles.

Session creator

- Creates a session around a task.
- Shares a join code with participants.
- Starts the analysis when enough responses have been submitted.
- Reviews the final agreement.

Participant

- Joins a session using its code.
- Submits an independent interpretation.
- Reviews potential misunderstandings after submission.
- Answers clarification questions.
- Confirms or requests changes to the final agreement.

For the hackathon, these roles do not require complex authentication. A participant can initially join using a display name and session code.

However, a join code should not be treated as strong authentication. A production version would require stronger identity and access controls.

4.2 Complete user journey

Step 1: Create a session

The creator enters:

- Task title
- Task description or original instruction
- Optional context
- Minimum number of participants, if configurable

Example:

Task: Prepare the application for launch.

Description: Finish the required functionality and prepare the application for release.

The backend creates a session with a unique identifier and human-readable join code.

Example join code: `GAP42X`

Step 2: Participants join

Other participants open the application and enter the join code.

They provide a display name and receive access to the task.

The application displays the number of participants who have joined and whether responses have been submitted.

Step 3: Collect independent interpretations

Each participant answers a prompt such as:

“In your own words, what do you think this task requires? What must be completed for it to count as finished?”

Optional prompts could include:

- What assumptions are you making?
- What is not included in the task?
- What does successful completion look like?
- What information do you still need?

The application stores each submission privately from other participants until the analysis phase begins.

Important: Independence is essential. If participants can read existing responses before submitting their own, their answers may be influenced by others and the system will have a less reliable comparison.

Step 4: Run AI analysis

When the creator starts the analysis, the backend collects the task description and all submitted interpretations.

The AI engine analyzes the responses and returns structured findings.

Possible findings include:

- Conflicting requirements
- Different assumptions
- Missing details
- Different definitions of completion
- Complementary ideas that can be combined

Step 5: Display the Misunderstanding Map

The interface presents each potential issue as a card.

Example:

Potential gap: Login requirements

- **Participant A:** “Users should log in using email and password.”
- **Participant B:** “Users should be able to log in using email or phone.”
- **Why this matters:** The team has not established whether phone login is part of the launch scope.
- **Clarification question:** “Should phone-number login be included in the first release?”
- **Priority:** High
- **Status:** Unresolved

The issue card should distinguish the original evidence from the AI's interpretation.

Step 6: Resolve issues

Participants review the clarification questions and select or enter an answer.

For example:

“The first release will support email and password only. Phone login is outside the initial scope.”

The system records this answer and updates the issue status.

Suggested statuses:

```
Open → Discussing → Resolved
```

Participants can also mark a finding as:

- Incorrect detection
- Not relevant
- Needs more information

Step 7: Generate a shared agreement

After clarification, the application generates a checklist based on the resolved issues and the original task.

Example:

Agreed launch requirements

- Email-and-password login must work.
- Registration must be available.
- Password recovery must be included.
- Phone-number login is excluded from the first release.
- The application must be usable on mobile devices.

Participants can confirm the checklist or request changes.

The agreement should not be marked as mutually confirmed until the required participants have explicitly confirmed it.

Step 8: Run the Bridge Test

The application compares the initial analysis with the state after clarification.

It shows:

- Initial number of potential issues
- Number of issues resolved
- Number of issues still unresolved
- Final confirmation status

For example, if the initial analysis identifies five issues and four are resolved, the application reports that four of five identified issues were resolved.

This is a measure of **resolution of detected issues**, not proof that all misunderstandings have been eliminated.

5. Functional requirements

The following are proposed implementation requirements for the project.

ID	Requirement	Priority
FR-01	Create a task session	Must have
FR-02	Generate a unique session code	Must have
FR-03	Allow participants to join using the code	Must have
FR-04	Accept and persist individual interpretations	Must have
FR-05	Prevent participants from seeing others' responses before submitting	Must have
FR-06	Trigger AI analysis of submitted responses	Must have

FR-07	Return structured findings with evidence	Must have
FR-08	Display findings in the Misunderstanding Map	Must have
FR-09	Generate clarification questions	Must have
FR-10	Record answers and update issue status	Must have
FR-11	Generate a shared checklist	Must have
FR-12	Allow participants to confirm the agreement	Must have
FR-13	Display before-and-after issue metrics	Should have
FR-14	Allow users to dismiss incorrect AI findings	Should have
FR-15	Support a polished demo session with sample data	Should have
FR-16	Add real-time updates using WebSockets	Could have

Recommendation: Implement FR-01 through FR-12 before spending time on real-time collaboration or advanced visualizations.

Polling or a refresh button is sufficient for the first demonstration.

6. Technical architecture

6.1 Recommended technology stack

For a three-person team working under a 24-hour deadline, use a stack that is fast to develop and straightforward to deploy.

Layer	Recommended technology	Purpose
Frontend	React	Build the interactive user interface
Build tooling	Vite	Run and build the frontend
Styling	Tailwind CSS	Create a consistent responsive interface
Backend	Python with FastAPI	Provide APIs and coordinate the workflow
Data validation	Pydantic	Validate API requests and AI output
Database	Supabase PostgreSQL	Store sessions, responses, issues, and agreements
AI engine	An available LLM API supporting structured output	Compare interpretations and generate questions
Hosting	Vercel or equivalent for frontend; a compatible backend host	Deploy the application

Source control	Git and GitHub	Collaborate and integrate code

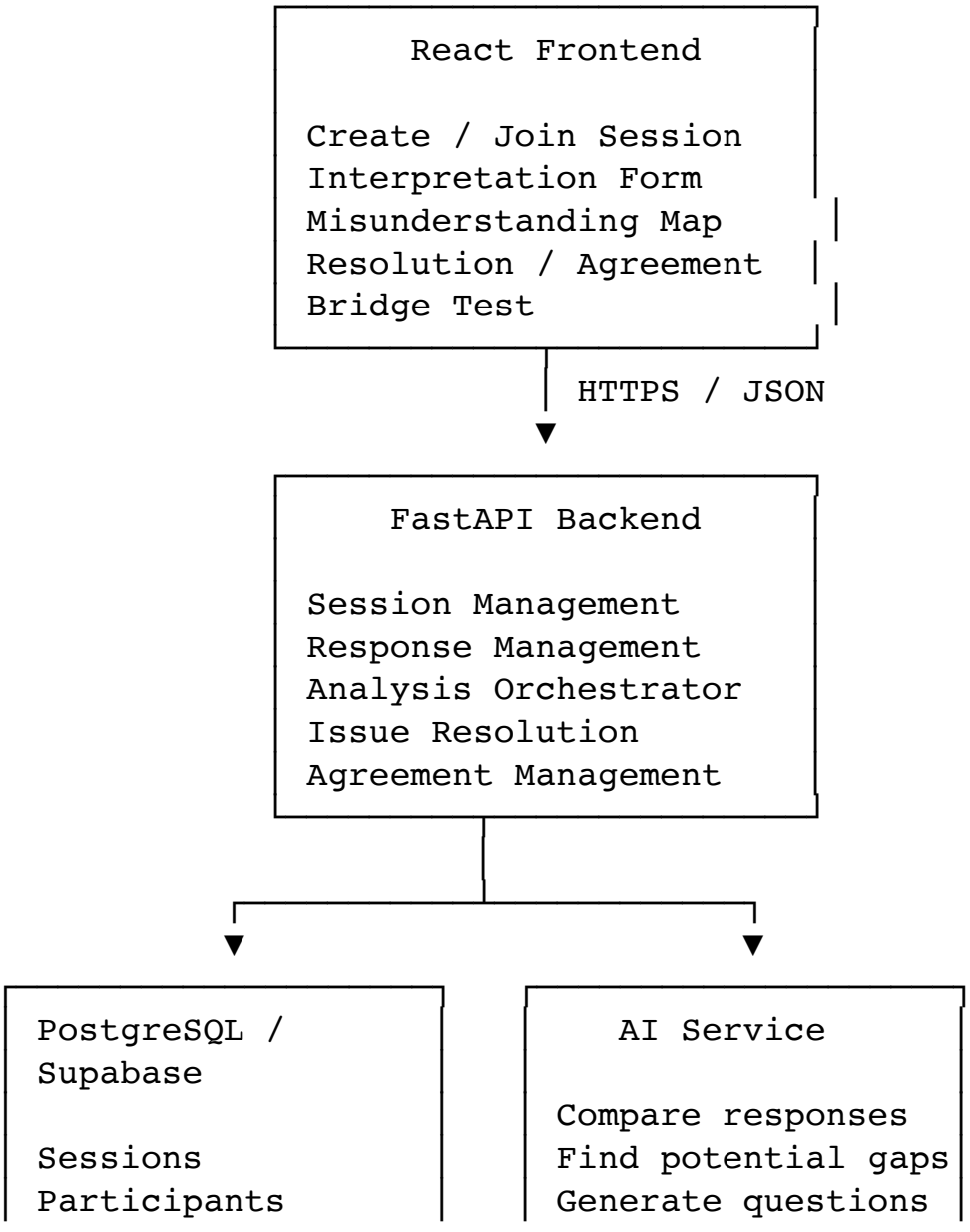
These are recommendations, not requirements imposed by the event documents.

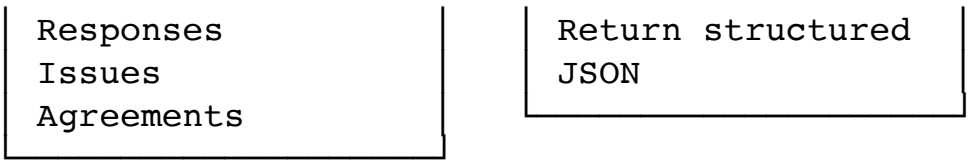
Why this stack?

- React supports a component-based UI.
- Vite keeps frontend setup lightweight.
- FastAPI makes it straightforward to expose Python APIs and validate structured data.
- PostgreSQL provides reliable persistence and relational constraints.
- An LLM API avoids spending the hackathon training a model.
- A managed database reduces infrastructure work.

If the team already has a familiar stack, prefer that over switching technologies during the hackathon.

6.2 High-level architecture





Request flow

1. The frontend sends a request to the backend.
2. The backend validates the request.
3. The backend reads or writes database records as needed.
4. When analysis is requested, the backend prepares the task and response data.
5. The backend sends that data to the AI service.
6. The backend validates the returned structure.
7. Valid findings are saved in the database.
8. The frontend retrieves the findings and renders them.

The frontend must never call the AI provider directly with a secret API key. All privileged API keys must remain on the server.

7. Core system components

7.1 Session management service

Responsibilities:

- Create sessions.
- Generate unique join codes.
- Validate join requests.
- Track session status.
- Ensure a participant cannot submit multiple active interpretations accidentally.

Suggested session states:

```
CREATED
   ↓
COLLECTING_RESPONSES
   ↓
ANALYZING
   ↓
REVIEWING_ISSUES
   ↓
AGREEMENT_PENDING
   ↓
COMPLETED
```

The exact state transitions should be enforced by the backend, not only by the frontend.

7.2 Interpretation service

Responsibilities:

- Accept individual responses.
- Validate that the participant belongs to the session.
- Store submitted text.
- Record submission timestamps.
- Prevent unauthorized reading of responses before the analysis stage.

The service should preserve the original text because the AI analysis needs to reference what participants actually said.

7.3 AI analysis service

This is the central intelligence component.

Responsibilities:

1. Prepare the task and submitted interpretations.
2. Identify potential semantic differences.
3. Categorize each issue.
4. Extract evidence from the submitted text.
5. Explain why each difference may matter.
6. Generate clarification questions.
7. Assign a tentative priority.
8. Return data matching a predefined schema.

The AI should not be treated as an unquestionable authority. Every finding must be reviewable by participants.

7.4 Misunderstanding Map service

The Misunderstanding Map is the principal results interface.

It groups findings into categories such as:

- Contradictions
- Unconfirmed assumptions
- Missing requirements
- Different completion criteria
- Complementary perspectives

Each finding should include its evidence, explanation, clarification question, priority, and resolution status.

7.5 Resolution service

Responsibilities:

- Store clarification answers.
- Update issue status.
- Preserve the resolution history.
- Generate a proposed agreement from the task and resolved issues.
- Record participant confirmations.

An issue should not become resolved merely because the AI generated an answer. The participants must accept the resolution.

7.6 Bridge Test service

The Bridge Test compares the state of the identified issues before and after clarification.

It should use saved analysis results and issue statuses rather than generating arbitrary success numbers.

For the MVP, simple counts and a resolution percentage are sufficient.

8. Database design

Use PostgreSQL with UUID primary keys. The following schema is a proposed design for the MVP.

8.1 `sessions`

Stores the task and collaboration session.

Column	Type	Description
`id`	UUID, primary key	Internal session identifier
`join_code`	VARCHAR, unique	Human-readable session code
`title`	VARCHAR	Task title
`description`	TEXT	Original task or instruction
`context`	TEXT, nullable	Additional context
`status`	VARCHAR	Current session state
`created_at`	TIMESTAMPTZ	Creation time
`updated_at`	TIMESTAMPTZ	Last update

Example status: `COLLECTING_RESPONSES`.

8.2 `participants`

Stores people participating in a session.

Column	Type	Description
`id`	UUID, primary key	Participant identifier
`session_id`	UUID, foreign key	Associated session
`display_name`	VARCHAR	Name shown in the interface

`role`	VARCHAR	Creator or participant
`joined_at`	TIMESTAMPTZ	Join time

For a minimal implementation, participant identity can be session-scoped. If users are represented only by display names, the application should not pretend that those names provide secure identity verification.

8.3 `interpretations`

Stores the independently submitted responses.

Column	Type	Description
`id`	UUID, primary key	Response identifier
`session_id`	UUID, foreign key	Associated session
`participant_id`	UUID, foreign key	Response author
`response_text`	TEXT	Main interpretation
`assumptions`	JSONB, nullable	Optional stated assumptions
`success_criteria`	JSONB, nullable	Optional completion criteria
`submitted_at`	TIMESTAMPTZ	Submission time

Recommended constraint: unique `(session_id, participant_id)` if each participant can submit only one interpretation per session.

8.4 `analyses`

Stores an AI analysis run.

Column	Type	Description
`id`	UUID, primary key	Analysis identifier
`session_id`	UUID, foreign key	Associated session
`model_name`	VARCHAR	AI model identifier
`prompt_version`	VARCHAR	Version of the analysis prompt
`status`	VARCHAR	Pending, completed, or failed
`result_json`	JSONB	Validated analysis result
`created_at`	TIMESTAMPTZ	Analysis start time
`completed_at`	TIMESTAMPTZ, nullable	Completion time

This table allows the application to retain a record of what analysis produced the initial findings.

8.5 `issues`

Stores individual potential misunderstandings.

Column	Type	Description
`id`	UUID, primary key	Issue identifier
`analysis_id`	UUID, foreign key	Analysis that identified the issue
`session_id`	UUID, foreign key	Associated session
`type`	VARCHAR	Contradiction, assumption, omission, etc.
`title`	VARCHAR	Short issue title
`description`	TEXT	Explanation
`evidence_json`	JSONB	Supporting response excerpts
`priority`	VARCHAR	Low, medium, or high
`clarification_question`	TEXT	Suggested question
`status`	VARCHAR	Open, discussing, resolved, dismissed
`resolution_text`	TEXT, nullable	Agreed resolution
`created_at`	TIMESTAMPTZ	Creation time
`resolved_at`	TIMESTAMPTZ, nullable	Resolution time

8.6 `agreements`

Stores the shared agreement.

Column	Type	Description
`id`	UUID, primary key	Agreement identifier
`session_id`	UUID, foreign key	Associated session
`version`	INTEGER	Agreement version
`content_json`	JSONB	Agreed checklist and scope
`status`	VARCHAR	Draft, pending confirmation, confirmed
`created_at`	TIMESTAMPTZ	Creation time

8.7 `agreement_confirmations`

Stores participant confirmations.

Column	Type	Description
`id`	UUID, primary key	Confirmation identifier
`agreement_id`	UUID, foreign key	Agreement being confirmed
`participant_id`	UUID, foreign key	Participant
`confirmed`	BOOLEAN	Whether the participant confirmed
`comment`	TEXT, nullable	Optional feedback
`confirmed_at`	TIMESTAMPTZ, nullable	Confirmation time

Recommended constraint: unique `(agreement_id, participant_id)`.

Relationship overview

```
sessions
    ├── participants
    │       ├── interpretations
    │       └── agreement_confirmations
    ├── analyses
    │       └── issues
    └── agreements
            └── agreement_confirmations
```

Database implementation notes

- Use foreign keys to prevent orphaned records.
- Add indexes to `sessions.join_code`, `participants.session_id`, `interpretations.session_id`, and `issues.session_id`.
- Use server-side timestamps.
- Validate status values using enums or database constraints.
- Never expose unrestricted database credentials to the browser.
- If using Supabase directly from the frontend, configure Row Level Security carefully. For the simplest hackathon architecture, route privileged operations through FastAPI.

9. AI analysis engine

9.1 Analysis input

The AI receives a structured input containing the task and participant responses.

Example:

```
{
  "task": {
    "title": "Prepare the application for launch",
```

```
    "description": "Finish required functionality before
release."
  },
  "participants": [
    {
      "participant_id": "participant_a",
      "interpretation": "Finish email and password login."
    },
    {
      "participant_id": "participant_b",
      "interpretation": "Finish email or phone login."
    },
    {
      "participant_id": "participant_c",
      "interpretation": "Complete onboarding and responsive
UI."
    }
  ]
}
```

The identifiers above are illustrative. In a real implementation, use internal participant IDs and avoid sending unnecessary personal information to the model.

9.2 Detection categories

The AI should use a small, controlled set of issue types.

Type	Meaning	Example
`contradiction`	Two statements appear incompatible	Email-only login versus email-or-phone login
`assumption`	A participant assumes a requirement not explicitly	Assuming phone login is mandatory
`omission`	An important detail is missing from the task or responses	No agreed definition of successful launch
`scope_difference`	Participants disagree about what is included	Password recovery is included for one person but excluded for another
`success_criteria_difference`	Participants define completion differently	One considers the UI finished; another expects a working backend
`complementary_perspective`	Different responses contribute useful, compatible information	One mentions accessibility while another mentions mobile

A complementary perspective is not necessarily a misunderstanding. It should be shown separately from conflicts so the application does not label every difference as a problem.

9.3 AI analysis prompt

Use a system prompt along these lines:

```
You are the analysis engine for MISUNDERSTOOD, a
collaboration tool
that helps people identify and resolve differences in their
interpretations of a shared task.

Your job is to compare the original task description with the
independent interpretations submitted by participants.

Identify only meaningful, evidence-supported findings.

Rules:

1. Treat the original task description and participant
responses
   as the source evidence.
2. Do not assume that every difference is a misunderstanding.
3. Distinguish genuine contradictions from compatible
differences
   in perspective.
4. Identify assumptions that have not been explicitly
confirmed.
5. Identify important missing requirements or undefined
success
   criteria when there is evidence that they matter.
6. For every issue, quote or closely reproduce the relevant
   participant statements.
7. Include participant identifiers for each piece of
evidence.
8. Explain why the issue could affect the task or outcome.
9. Generate one clear, neutral clarification question per
issue.
10. Do not decide which participant is correct.
11. Do not invent participant statements or requirements.
12. If the evidence is insufficient, omit the issue or mark
it
    as uncertain.
13. Assign a tentative priority based on the likely impact on
the
    task, not on the identity of the participant.
14. Return only valid JSON matching the required schema.
```

The prompt is a proposed implementation. The team should test it against several sample tasks and adjust it based on the quality of its findings.

9.4 Expected AI output schema

For the MVP, use a predictable JSON structure.

```
{
  "summary": "Participants have different expectations about
launch scope.",
  "issues": [
    {
      "type": "scope_difference",
      "title": "Conflicting login requirements",
      "description": "The participants have different
expectations about supported login methods.",
      "evidence": [
        {
          "participant_id": "participant_a",
          "quote": "Finish email and password login."
        },
        {
          "participant_id": "participant_b",
          "quote": "Finish email or phone login."
        }
      ],
      "impact": "The team could implement different login
behavior.",
      "priority": "high",
      "confidence": 0.92,
      "clarification_question": "Should the first release
support email login only, or email and phone login?"
    }
  ],
  "complementary_perspectives": [
    {
      "title": "Responsive design",
      "description": "The mobile-responsive interface may
complement the functional requirements.",
      "evidence": [
        {
          "participant_id": "participant_c",
          "quote": "Complete onboarding and responsive UI."
        }
      ]
    }
  ]
}
```

The confidence value in this example is illustrative. A language model's self-reported confidence should not be interpreted as a calibrated probability that an issue is real.

Required validation

The backend must verify that:

- The response is valid JSON.
- Every issue has an allowed type.
- Every priority uses an allowed value.
- Every issue contains a title, description, evidence, and question.
- Evidence refers to actual submitted participants.
- Evidence excerpts can be matched to the source responses, preferably through server-generated evidence references.
- The output does not exceed reasonable length limits.
- The number of returned issues is capped.

If the AI output fails validation, the backend should retry once with a repair instruction or return a recoverable error. It should not silently save malformed findings.

9.5 Evidence references

A strong implementation links each AI finding back to the exact source response.

For example:

```
{
  "participant_id": "participant_a",
  "interpretation_id": "response_123",
  "quote": "Finish email and password login."
}
```

The backend should generate the `interpretation_id`; the AI should not invent database IDs.

This improves traceability and makes the Misunderstanding Map more trustworthy.

10. How misunderstanding detection works

The system does not need a custom machine-learning model. The MVP can use an LLM with a carefully designed prompt and structured output.

A more reliable implementation uses several stages.

Stage 1: Validate the inputs

Check that the session exists, the task is available, and enough participants have submitted responses.

For example, the analysis can require at least two completed interpretations.

Stage 2: Normalize the task and responses

Prepare the task description and participant responses in a consistent structure.

Preserve the original wording. Do not rewrite the responses in a way that changes their meaning.

Stage 3: Identify candidate gaps

Ask the LLM to compare the interpretations and propose candidate issues.

The model should look for differences that could affect:

- Scope
- Behavior
- Deliverables
- Constraints
- Completion criteria
- Responsibilities
- Dependencies
- Assumptions

Stage 4: Validate the evidence

For each candidate issue, check whether the supporting excerpts actually appear in the relevant responses.

If the evidence cannot be verified, reject the finding or request a revised result.

This does not prove that the interpretation is correct, but it helps prevent fabricated quotations.

Stage 5: Assign tentative priority

A simple initial priority rubric can be:

- **High:** The difference could lead to incompatible implementations, major rework, or a failed outcome.
- **Medium:** The difference could cause delays, confusion, or a meaningful mismatch.
- **Low:** The difference is minor, cosmetic, or unlikely to affect the outcome.

Priority is an estimate, not a measured fact.

Stage 6: Generate clarification questions

Each issue should produce one question that helps the participants make a decision.

Weak question:

“Can you clarify the login requirements?”

Better question:

“Should the first release support email login only, or email and phone login?”

A good question presents the actual unresolved decision and avoids blaming a participant.

Stage 7: Present the findings

Display the findings in an interactive interface where participants can inspect evidence, answer questions, and mark incorrect detections.

Important design principle

A difference is a signal to investigate, not proof of a misunderstanding.

The application should avoid overconfident language such as “Participant B is wrong.” Prefer language such as “Potential scope difference” or “This requirement has not been confirmed.”

11. API design

Use a REST API with JSON request and response bodies.

The following endpoints are proposed for the MVP.

Method	Endpoint	Purpose
`POST`	`/api/sessions`	Create a session
`GET`	`/api/sessions/{session_id}`	Get session details
`POST`	`/api/sessions/join`	Join using a code
`GET`	`/api/sessions/{session_id}/`	List participants
`POST`	`/api/sessions/{session_id}/interpretations`	Submit an interpretation
`GET`	`/api/sessions/{session_id}/status`	Check submission and analysis status
`POST`	`/api/sessions/{session_id}/analyze`	Start AI analysis
`GET`	`/api/sessions/{session_id}/issues`	Retrieve issues
`PATCH`	`/api/issues/{issue_id}`	Update issue status or resolution
`POST`	`/api/sessions/{session_id}/agreement`	Generate or save an agreement
`POST`	`/api/agreements/{agreement_id}/`	Confirm an agreement
`GET`	`/api/sessions/{session_id}/bridge-`	Retrieve before-and-after

11.1 Create a session

Request

```
POST /api/sessions

{
```

```
  "title": "Prepare the application for launch",
  "description": "Finish the required functionality before
release.",
  "context": "The first release is intended for web users."
}
```

Response

```
{
  "id": "session_uuid",
  "join_code": "GAP42X",
  "status": "COLLECTING_RESPONSES"
}
```

The example IDs and join code are placeholders.

11.2 Join a session

Request

```
POST /api/sessions/join

{
  "join_code": "GAP42X",
  "display_name": "Participant A"
}
```

Response

```
{
  "participant_id": "participant_uuid",
  "session_id": "session_uuid",
  "status": "COLLECTING_RESPONSES"
}
```

If anonymous participation is used, the backend should issue a secure participant token or session credential. A plain participant ID is not sufficient authorization by itself.

11.3 Submit an interpretation

Request

```
POST /api/sessions/{session_id}/interpretations

{
  "participant_id": "participant_uuid",
  "response_text": "Finish email and password login.",
  "assumptions": [
    "Phone login is not required."
  ],
  "success_criteria": [
```

```
    "Users can log in successfully."
  ]
}
```

Response

```
{
  "interpretation_id": "response_uuid",
  "submitted": true,
  "message": "Your interpretation has been saved."
}
```

The backend must verify the participant's authorization and ensure the session is accepting submissions.

11.4 Start AI analysis

Request

```
POST /api/sessions/{session_id}/analyze
```

Response

```
{
  "analysis_id": "analysis_uuid",
  "status": "ANALYZING"
}
```

For the MVP, analysis can be synchronous if it reliably completes within the HTTP timeout. An asynchronous job is preferable if the AI provider may take longer.

11.5 Retrieve issues

Request

```
GET /api/sessions/{session_id}/issues
```

Response

```
{
  "session_id": "session_uuid",
  "issues": [
    {
      "id": "issue_uuid",
      "type": "scope_difference",
      "title": "Conflicting login requirements",
      "priority": "high",
      "status": "open",
      "clarification_question": "Should phone login be
included?",
      "evidence": [
```

```
        {
          "participant_id": "participant_a",
          "quote": "Finish email and password login."
        },
        {
          "participant_id": "participant_b",
          "quote": "Finish email or phone login."
        }
      ]
    }
  ]
}
```

11.6 Resolve an issue

Request

```
PATCH /api/issues/{issue_id}

{
  "status": "resolved",
  "resolution_text": "The first release will support email
and password only."
}
```

Response

```
{
  "id": "issue_uuid",
  "status": "resolved",
  "resolution_text": "The first release will support email
and password only."
}
```

In a real implementation, the backend must check the caller's permission to resolve the issue.

11.7 Generate an agreement

Request

```
POST /api/sessions/{session_id}/agreement
```

Response

```
{
  "agreement_id": "agreement_uuid",
  "status": "PENDING_CONFIRMATION",
  "items": [
    {
```

```
      "text": "The first release will support email and
password login.",
      "source": "resolved_issue"
    },
    {
      "text": "Phone-number login is excluded from the first
release.",
      "source": "resolved_issue"
    }
  ]
}
```

Only requirements supported by the original task, submitted responses, or explicit resolutions should be included. Any newly suggested requirement should be labeled as a suggestion until accepted.

11.8 Retrieve the Bridge Test

Request

```
GET /api/sessions/{session_id}/bridge-test
```

Response

```
{
  "initial_issue_count": 5,
  "resolved_issue_count": 4,
  "unresolved_issue_count": 1,
  "dismissed_issue_count": 0,
  "resolution_rate_percent": 80,
  "agreement_status": "PENDING_CONFIRMATION"
}
```

The values above are illustrative. Real values must be computed from the stored analysis and issue records.

12. Shared agreement generation

The agreement is the outcome of the resolution workflow.

It should not simply be a summary of everything participants said. It should distinguish between three categories:

1. **Confirmed requirements:** Participants have explicitly agreed.
2. **Unresolved decisions:** Participants still need to decide.
3. **Optional suggestions:** AI-generated ideas that have not yet been accepted.

Example:

Confirmed requirements

- Email-and-password login will be included.
- Registration must work.
- The interface must be usable on mobile devices.

Unresolved decisions

- Is password recovery required for the initial release?

Optional suggestion

- Add an explicit definition of “ready for launch” to the project checklist.

This separation is important because AI-generated content should not silently become a team commitment.

Confirmation logic

The agreement can move through the following states:

```
DRAFT
  ↓
PENDING_CONFIRMATION
  ↓
CONFIRMED
```

If a participant requests changes, create a new version or return the agreement to the draft state.

For the MVP, the application can require confirmation from every participant who submitted an interpretation. If that rule is used, it should be visible to users and enforced by the backend.

13. The Bridge Test

The Bridge Test is the project's demonstration of impact.

It compares the initial set of AI-detected issues with the issue statuses after participants have clarified their understanding.

13.1 Core metrics

Let:

- N_0 = number of issues identified in the initial analysis
- R = number of those issues resolved
- U = number of those issues still unresolved
- D = number dismissed as incorrect or irrelevant

If every initial issue is accounted for, then:

$$ N_0 = R + U + D $$

The resolution rate can be calculated as:

$$ \text{Resolution Rate} = \frac{R}{N_0} \times 100 $$

For example, if five issues were initially detected and four were resolved:

$$ \text{Resolution Rate} = \frac{4}{5} \times 100 = 80\% $$
If the initial analysis detects zero issues, the percentage should be displayed as “Not applicable” rather than dividing by zero.

13.2 Suggested interface

Display four metrics:

- **Potential gaps found**
- **Issues resolved**
- **Issues remaining**
- **Agreement confirmation**

A simple before-and-after visualization can show the initial findings alongside the final issue statuses.

What the Bridge Test can and cannot claim

It can show how many detected issues were resolved and whether the participants confirmed the final agreement.

It cannot prove that the team has no remaining misunderstandings. The initial AI analysis may have missed issues, and participants may agree to an incomplete or incorrect requirement.

For a 24-hour hackathon, these transparent metrics are preferable to an unsupported “understanding score” that claims to measure people's actual understanding.

14. Frontend design and screens

The frontend should make the project understandable within a few seconds. The user should immediately see the communication problem, the gap-detection mechanism, and the result of clarification.

Screen 1: Landing page

Headline: MISUNDERSTOOD

Subheading: “Same conversation. Different realities.”

Primary actions:

- Create a session
- Join a session

The landing page should briefly explain that participants submit their own interpretations and the system helps them identify differences.

Screen 2: Create session

Fields:

- Task title
- Task description
- Optional context

Action:

- Create session

After creation, display the join code prominently.

Screen 3: Waiting room

Display:

- Task title
- Join code
- Participant count
- Submission progress
- Start-analysis button

The start-analysis button should be disabled until the minimum number of participants has submitted a response.

Screen 4: Interpretation form

Display the original task and a response form.

Suggested questions:

1. What do you think this task requires?
2. What assumptions are you making?
3. What would count as completion?

Allow users to submit their answers independently.

Screen 5: Misunderstanding Map

This is the main results screen.

Each issue card should show:

- Issue type
- Title
- Evidence from each participant
- Explanation
- Priority
- Clarification question
- Current status
- Resolve or dismiss action

Use clear visual hierarchy rather than overwhelming users with charts.

Screen 6: Clarification and resolution

Display the selected issue and its evidence.

Participants enter or select a resolution. The backend saves the decision and updates the issue status.

Screen 7: Shared agreement

Show the proposed checklist with clear labels for confirmed, unresolved, and suggested items.

Allow participants to confirm the agreement.

Screen 8: Bridge Test

Display the initial issue count, resolved issues, remaining issues, and confirmation status.

A concise message might say:

“You resolved 4 of the 5 potential gaps identified in the initial analysis.”

This wording accurately describes what the app measured.

15. Security, privacy, and reliability

Even a hackathon prototype should follow basic security principles.

15.1 API key protection

- Store AI-provider keys in backend environment variables.
- Do not commit `.env` files to Git.
- Do not include secret keys in frontend code.
- Configure separate environment variables for local development and deployment.

15.2 Input validation

The backend should:

- Require a non-empty task title and description.
- Limit response length.
- Validate participant and session IDs.
- Reject invalid state transitions.
- Validate AI output before saving it.
- Rate-limit session creation and expensive analysis requests where practical.

15.3 Session access

A session code is a convenience mechanism, not strong authentication.

For the prototype:

- Use sufficiently random join codes.
- Require a valid session code to join.
- Avoid exposing participant responses before submission.

- Check access on every session and issue endpoint.
- Prevent participants from accessing unrelated sessions by changing IDs in requests.

For a production system, use proper authentication, authorization, session expiration, and secure participant credentials.

15.4 Privacy

Interpretations may contain sensitive team or project information.

The application should:

- Collect only the information necessary for the task.
- Explain that responses are sent to an external AI service if applicable.
- Avoid transmitting unnecessary personal details.
- Avoid logging full response text or API secrets unnecessarily.
- Provide a way to delete a session and its associated data.
- Document how long data is retained.

15.5 AI failure handling

The system should handle:

- Provider timeouts
- Rate limits
- Invalid model output
- Empty responses
- Database errors
- Duplicate analysis requests
- Partial analysis failures

A failed analysis should not destroy submitted interpretations.

Recommended behavior:

1. Save participant responses before analysis.
2. Create an analysis record.
3. Mark the analysis as pending or running.
4. Call the AI provider.
5. Validate and persist successful results.
6. Mark the analysis completed.
7. If it fails, mark it failed and show a retry action.

Use a request identifier or other idempotency mechanism to avoid accidentally creating duplicate analyses when users click the button repeatedly.

16. Testing strategy

The team should test the core workflow before adding optional features.

16.1 Functional tests

Test	Expected result
Create a valid session	Session and join code are returned
Join with a valid code	Participant joins successfully
Join with an invalid code	Backend returns an error
Submit an interpretation	Response is saved
Submit an empty response	Backend rejects it
View responses before submission	Other participants' responses remain hidden
Analyze two different interpretations	Findings are returned and validated
Analyze similar interpretations	No forced contradiction is created
Resolve an issue	Status and resolution are persisted
Dismiss an incorrect issue	It is no longer counted as unresolved
Generate an agreement	Checklist reflects the saved task and resolutions
Confirm an agreement	Confirmation is recorded
Run Bridge Test	Metrics match stored issue states
AI provider fails	Responses remain saved and retry is available

16.2 AI evaluation examples

Prepare at least three sample sessions.

Scenario A: Clear contradiction

- Participant A: “The app must support email login only.”
- Participant B: “The app must support email and phone login.”

Expected: a possible scope conflict with a question about supported login methods.

Scenario B: Complementary perspectives

- Participant A: “The app should be mobile responsive.”
- Participant B: “The app should have accessible text and controls.”

Expected: these should normally be treated as complementary requirements, not contradictory ones.

Scenario C: Missing requirement

- Participant A: “Build the registration page.”
- Participant B: “Complete registration and login.”
- Original task: “Prepare authentication for launch.”

Expected: the system may ask whether password recovery, validation, or another requirement is needed—but should not assert that any specific missing feature is mandatory without supporting context.

16.3 Demo acceptance criteria

The MVP is ready to demonstrate when:

1. A creator can create a session.
2. Two participants can submit independent interpretations.
3. The system can analyze those responses.
4. At least one meaningful issue can be displayed with traceable evidence.
5. A participant can answer a clarification question.
6. The issue status updates.
7. The application generates a shared checklist.
8. The Bridge Test displays metrics derived from actual records.

Use a known sample scenario to validate the flow before the event demo. The sample should be used to test the system, not as a substitute for implementing the analysis and resolution workflow.

17. Deployment plan

17.1 Environment configuration

Example backend environment variables:

```
DATABASE_URL=
AI_API_KEY=
AI_MODEL=
FRONTEND_ORIGIN=
```

These are placeholders; the team should use the actual configuration names required by its chosen database and AI provider.

Do not commit real values.

17.2 Frontend deployment

1. Build the React application.
2. Set the public backend API URL.
3. Deploy the frontend to a suitable hosting service.
4. Verify the deployed site loads correctly on mobile and desktop.

Only public configuration should be placed in frontend environment variables.

17.3 Backend deployment

1. Install the Python dependencies.
2. Configure database access.
3. Configure the AI API key.
4. Configure allowed frontend origins.
5. Start the FastAPI application.
6. Verify the health endpoint.
7. Test session creation, response submission, and analysis against the deployed database.

17.4 Pre-demo checklist

- Frontend URL opens successfully.
- Backend health check succeeds.
- Database connection works.
- AI API key works.
- A session can be created.
- At least two participants can join.
- Both participants can submit responses.
- Analysis produces valid findings.
- Evidence is visible in the Misunderstanding Map.
- An issue can be resolved.
- The agreement can be generated.
- Bridge Test metrics match the saved issues.
- Error handling works if AI analysis fails.
- Demo accounts and sample responses are prepared.

18. Division of work among three team members

The team should divide work by system component, but agree on the API contracts and JSON schema before implementing everything independently.

Member 1 — Frontend and user experience

Primary responsibility: Build the complete user journey.

Tasks:

- Create the React/Vite project.
- Design the landing page.
- Build the create/join session interface.
- Implement the independent interpretation form.
- Build the Misunderstanding Map.
- Build issue-resolution controls.
- Build the shared-agreement screen.
- Build the Bridge Test.
- Connect all screens to backend APIs.
- Handle loading states and API errors.

Deliverable: A deployed or locally runnable frontend connected to the backend.

Member 2 — Backend and database

Primary responsibility: Build the application infrastructure and workflow.

Tasks:

- Set up FastAPI.
- Create database tables.
- Implement session creation and joining.
- Implement interpretation submission.

- Implement access checks.
- Integrate the AI service with Member 3's module.
- Implement issue retrieval and resolution.
- Implement agreement creation and confirmation.
- Implement Bridge Test metrics.
- Deploy and test the backend.

Deliverable: A functioning API with persisted sessions, responses, issues, and agreements.

Member 3 — AI and analysis engine

Primary responsibility: Build and test the misunderstanding-detection logic.

Tasks:

- Select and configure the LLM API.
- Design the system prompt.
- Define the Pydantic output models.
- Implement structured response parsing and validation.
- Detect potential contradictions and scope differences.
- Identify assumptions and missing requirements.
- Generate clarification questions.
- Attach evidence references to real participant responses.
- Test the AI against positive and negative examples.
- Tune the output to reduce false positives.

Deliverable: A backend-callable analysis module that accepts task data and returns validated JSON.

Shared responsibility

All three members should:

- Agree on the API contract.
- Use Git branches and frequent integration.
- Test the complete workflow together.
- Prepare a known-good demonstration scenario.
- Rehearse the pitch and functional walkthrough.

Important: Member 2 and Member 3 must integrate early. The AI module should not be developed in isolation until the end of the hackathon.

19. Recommended 24-hour implementation plan

The event schedule document specifies a first build round from **11:00 AM to 4:00 PM**, a mentor checkpoint and shortlisting period from **4:00 PM to 5:30 PM**, and an overnight build period after shortlisted teams resume at **6:00 PM**. The document lists the final build lock at **9:00 AM on Day 2**.

Plan the implementation around those checkpoints.

Phase	Approximate time	Goal

1. Define contract	Hours 0–1	Agree on schema, endpoints, UI flow,
2. Skeleton build	Hours 1–3	Frontend, backend, and database
3. Session workflow	Hours 3–5	Create/join session and submit
4. Mentor checkpoint preparation	Before 4 PM checkpoint, according to event schedule	Demonstrate the core idea with an early working prototype
5. AI integration	Hours 5–9	Analyze responses and display
6. Resolution	Hours 9–13	Clarification answers and issue status
7. Agreement	Hours 13–16	Generate and confirm the shared
8. Bridge Test	Hours 16–18	Calculate metrics from stored issues
9. Integration and	Hours 18–21	Fix end-to-end failures and edge cases
10. Polish and	Hours 21–23	Improve the UI, prepare demo data,
11. Final validation	Hours 23–24	Freeze features and verify the final

This is a proposed engineering plan, not a replacement for the event's official schedule.

First milestone: the minimum working prototype

Before building advanced features, ensure the team can demonstrate:

1. A session is created.
2. Two users submit different interpretations.
3. AI identifies a plausible gap.
4. The application shows the evidence.
5. A clarification answer resolves the issue.

That is the smallest convincing demonstration of the project's central idea.

20. Main risks and mitigation

Risk	Why it matters	Mitigation
AI invents a	Users may lose trust in the	Require evidence and allow dismissal
AI output is invalid	The frontend may fail to	Use structured output and Pydantic validation
AI API is slow or	The demo may stall	Add timeouts, retry handling, and a tested
Frontend and backend contracts differ	Integration becomes difficult	Agree on schemas before parallel development
Database access is	Participants may access	Enforce server-side authorization
Scope grows too large	Core workflow may	Prioritize the MVP and postpone optional
Bridge Test claims too much	Metrics could be misleading	Report detected and resolved issues, not a universal understanding score
Team members see answers too early	Independent interpretations become	Enforce the submission gate on the backend

Fallback plan

If the AI service is unavailable during the demonstration, the team should have a known-good analysis result that was generated and validated earlier.

Use the fallback only with a clear explanation or as a recovery mode. The primary demo should show the actual working application and its analysis workflow, rather than pretending prerecorded output was generated live.

This is especially important because the event design document emphasizes execution and a functional demo, not a UI-only simulation. Event Design Devlympics

21. Demonstration scenario

Use a single, clear scenario to demonstrate the full product.

Initial task

“Prepare the application for launch.”

Participant A

“Finish email-and-password login and registration.”

Participant B

“Finish email or phone login, registration, and password recovery.”

Participant C

“Complete onboarding and make the interface responsive on mobile.”

What the AI should identify

Potential issue 1: Login scope

Participants A and B describe different login requirements.

Clarification question:

“Should phone login be included in the first release?”

Potential issue 2: Password recovery

Participant B includes password recovery, but the other responses do not establish whether it is required.

Clarification question:

“Is password recovery a launch requirement, or is it outside the initial scope?”

This should be presented as an unconfirmed requirement, not automatically as a contradiction.

Potential issue 3: Definition of launch readiness

The task does not define what it means for the application to be ready for launch.

Clarification question:

“Which features and acceptance criteria must be completed before the team considers the application ready for launch?”

After clarification

The team agrees:

- Email-and-password login is required.
- Phone login is excluded from the initial release.
- Password recovery is required.
- The interface must be responsive.
- The team will use an agreed checklist to define launch readiness.

The application generates the shared agreement and updates the Bridge Test.

This scenario demonstrates the complete concept: **different interpretations become visible, uncertainties become explicit decisions, and decisions become a shared plan.**

22. How to make the project stand out

The project should not rely on the claim that AI can compare text. The stronger story is the complete process from independent understanding to confirmed agreement.

Recommended differentiators

1. Independent-first collaboration

Participants submit their interpretations before seeing others' responses.

2. Evidence-backed findings

Every issue links to what participants actually wrote.

3. Clarification instead of blame

The application asks neutral, targeted questions rather than deciding who is wrong.

4. Human-confirmed agreement

The system records what the team actually accepts instead of treating AI output as a decision.

5. The Bridge Test

The application shows what happened to the identified issues after clarification.

6. A memorable experience

Use a visually engaging Misunderstanding Map, clear issue cards, and a concise before-and-after view. Any playful elements should reinforce the concept without making the analysis look unreliable.

The event design document specifically evaluates theme interpretation, fun and experience, creativity, execution, presentation, and participation in event twists. The team should therefore demonstrate a real end-to-end workflow and present it clearly, rather than spending all its time on extra features.

23. Final MVP specification

For the 24-hour build, the recommended MVP is:

Required features

- Create and join a session.
- Collect independent interpretations from at least two participants.
- Store the responses.
- Analyze them with an LLM.
- Identify potential differences with supporting evidence.
- Generate clarification questions.
- Resolve or dismiss issues.
- Generate a shared checklist.
- Calculate Bridge Test metrics.

Optional features

- Real-time updates.
- Advanced graphs.
- Multiple agreement versions.
- Export to PDF.
- Integration with project-management tools.
- Voice input.
- Long-term team history.
- User accounts and persistent profiles.

Do not add optional features until the required workflow works reliably.

Definition of done

The project is complete enough for its core demonstration when a judge can create a task, submit two different interpretations, see a meaningful evidence-backed gap, resolve it, and view the resulting agreement and Bridge Test.

The final demo should show real application behavior, not just static screens.

24. Final technical summary

MISUNDERSTOOD is a web application built around a simple but important observation: **people can agree on the words of a task without agreeing on what those words mean.**

The proposed architecture combines:

- **React** for the user interface.
- **FastAPI** for backend APIs and workflow management.
- **PostgreSQL/Supabase** for persistent data.
- **An LLM API** for semantic comparison and clarification-question generation.
- **Structured JSON validation** for reliable integration.
- **Evidence-linked issue cards** for transparency.
- **Human confirmation** for the final agreement.
- **The Bridge Test** for measuring the resolution of detected issues.

Its central technical contribution is not merely identifying text differences. It is connecting independent interpretation, evidence-backed analysis, targeted clarification, and a confirmed shared agreement into one usable workflow.

The core principle is:

Don't just ask people whether they understand. Compare what they understood, clarify what differs, and record what they finally agree on.